

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S1
DATE PERFORMED:	July 26, 2026	DATE SUBMITTED:	Jul 28, 2026
NAME:	Mendoza, Ryo Shin P.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 LINKED LIST	
1. PROCEDURE OUTPUT	
ILO A: Construct C++ code for a singly and doubly linked list in C++:	
Screenshot	<pre> 1 #include <iostream> 2 3 class Node 4 { 5 public: 6 char data; 7 Node *next; 8 }; 9 10 int main() 11 { 12 // step 1: Create the node pointing to null 13 Node *head = NULL; 14 Node *second = NULL; 15 Node *third = NULL; 16 Node *fourth = NULL; 17 Node *fifth = NULL; 18 Node *last = NULL; 19 20 // step 2: 21 head = new Node; 22 second = new Node; 23 third = new Node; 24 fourth = new Node; 25 fifth = new Node; 26 last = new Node; 27 28 second->data = 'P'; 29 second->next = third; 30 31 third->data = 'E'; 32 third->next = fourth; 33 34 fourth->data = '0'; 35 fourth->next = fifth; 36 37 fifth->data = '1'; 38 fifth->next = last; 39 40 last->data = '0'; 41 last->next = nullptr; 42 43 // [C] -> [P] -> [E] -> [0] -> [1] -> [0] -> NULL 44 } </pre>
Discussion	The output C -> P -> E -> 0 -> 1 -> 0 -> NULL is produced by creating six nodes, storing one character in each node, and linking them together using the next pointer. The program starts at the head node and follows each next pointer until it reaches NULL, printing the character stored in every node. This forms the linked list that represents the string "CPE010". The program can be improved by using loops and functions instead of creating each node manually. Also, provide a deallocation to prevent memory leaks.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	<pre>// Display the linked list (Transversal) Node *current = head; while (current != nullptr) { cout << current->data << " -> "; current = current->next; } cout << "NULL";</pre> Output: C -> P -> E -> 0 -> 1 -> 0 -> NULL
Insertion at the head	<pre>// Insertion at the head // Step 1: Allocate memory for the new node Node *newNode = new Node; // Step 2: Put data into the new node newNode->data = 'C'; // Step 3: Point the new node to the previous head newNode->next = head; // Step 4: Reset head to the new node head = newNode;</pre>
Insertion at any part	<pre>// Create the new node Node *newNode = new Node; newNode->data = 'X'; // Traverse to the node after which you want to insert Node *current = head; while (current->data != 'E') { current = current->next; } // Insert the new node newNode->next = current->next; current->next = newNode;</pre>

Insertion at the end	<pre>// Insert at the end // Step 1: Create the new node Node *newNode = new Node; newNode->data = '1'; newNode->next = nullptr; // Step 2: Traverse to the last node Node *current = head; while (current->next != nullptr) { current = current->next; } // Step 3: Link the last node to the new node current->next = newNode;</pre>
Deletion of the node	<pre>// Delete the node containing 'E' Node *current = head; Node *previous = nullptr; while (current != nullptr && current->data != 'E') { previous = current; current = current->next; } if (current != nullptr) { previous->next = current->next; delete current; }</pre>

Table 3-2. Code for the List Operations

a.	Source Code	<pre> template <typename T> void listTraversal(Singlelist<T>* head) { // Continue until we reach the end of the list while (head != nullptr) { // Print the current node's data std::cout << head->data; // Print an arrow if another node follows if (head->next != nullptr) { std::cout << " --> "; } // Move to the next node head = head->next; } // Move to the next line after printing the list std::cout << std::endl; } // a. Traverse the list cout << "(a) Traverse the original list:" << endl; traverse(head); cout << endl; </pre>
	Console	<pre> (a) Traverse the original list: CPE101 </pre>

b.	Source Code	<pre>// Insert at the start void insertStart(Node*& head, char value) { Node* newNode = new Node; newNode->data = value; newNode->next = head; head = newNode; } // b. Insert G at the start cout << "(b) Insert G at the start:" << endl; insertStart(head, 'G'); traverse(head); cout << endl;</pre>
	Console	<pre>(b) Insert G at the start: GCPE101</pre>

c.	Source Code	<pre>// Insert after a specific node void insertAfter(Node* previous, char value) { if (previous == nullptr) return; Node* newNode = new Node; newNode->data = value; newNode->next = previous->next; previous->next = newNode; } // c. Insert E after P cout << "(c) Insert E after P:" << endl; Node *current = head; while (current != nullptr && current->data != 'P') current = current->next; insertAfter(current, 'E'); traverse(head); cout << endl;</pre>
	Console	<pre>(c) Insert E after P: GCPEE101</pre>

d.	Source Code	<pre> void deleteNode(Node*& head, char value) { Node* current = head; Node* previous = nullptr; // If deleting the first node if (current != nullptr && current->data == value) { head = current->next; delete current; return; } // Search for the node while (current != nullptr && current->data != value) { previous = current; current = current->next; } // Delete if found if (current != nullptr) { previous->next = current->next; delete current; } } </pre> <pre> // d. Delete C cout << "(d) Delete C:" << endl; deleteNode(head, 'C'); traverse(head); cout << endl; </pre>
	Console	<pre> (d) Delete C: GPEE101 </pre>

e.	Source Code	<pre>// e. Delete P cout << "(e) Delete P:" << endl; deleteNode(head, 'P'); traverse(head); cout << endl;</pre> Note: It has a same function deleting C
	Console	<pre>(e) Delete P: GEE101</pre>
f.	Source Code	<pre>// f. Display final list cout << "(f) Final list:" << endl; traverse(head);</pre> Note: Same function used as letter a.
	Console	<pre>(f) Final list: GEE101</pre>

Table 3-3. Code and Analysis for Singly Linked Lists

A.2: Doubly Linked List

Screenshots	Analysis
1. Traverse the Linked List <pre>template <typename T> void dllTraverse(doubleList<T>* currentNode) { doubleList<T>* tail = nullptr; while (currentNode != nullptr) { std::cout << currentNode->data << " "; tail = currentNode; currentNode = currentNode->next; } std::cout << std::endl; // b. Insert G at the start cout << "(b) Insert G at the start:" << endl; insertStart(head, 'G'); traverse(head); cout << endl; Initial Traversal C P E </pre>	The program traverses the doubly linked list from the head to the tail. Each node is connected using both next and prev pointers, allowing movement in both directions while displaying the elements in order.
2. Insert at the Head <pre>template <typename T> void dllInsertHead(T newData, doubleList<T>** head) { doubleList<T>* newNode = new_node(newData); newNode->next = *head; if (*head != nullptr) { (*head)->prev = newNode; } *head = newNode; } // Insert at Head std::cout << "\nInsert at Head:" << std::endl; dllInsertHead('G', &head); dllTraverse(head); Insert at Head G C P E </pre>	A new node containing G is inserted before the current head. The new node's next pointer points to the old head, the old head's prev pointer points back to the new node, and the head pointer is updated.

3. Insert a new element at the end

```
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* head) {

    if (head == nullptr)
        return;

    doubleList<T>* newNode = new_node(newData);

    doubleList<T>* current = head;

    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = newNode;
    newNode->prev = current;
}
```

```
// Insert at End
std::cout << "\nInsert at End:" << std::endl;
dllInsertEnd('1', head);
dllTraverse(head);
```

Insert at End:
G C P E 1

A new node containing the value 1 is added at the end of the doubly linked list. The program traverses the list until it reaches the last node, then updates the last node's next pointer to the new node.

4. General Insertion

```
template <typename T>
void dllGeneralInsertion(doubleList<T>* prevNode, T newData) {

    //Check if the previous node exist
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

    //Create a new node
    doubleList<T>* newNode = new_node(newData);

    // Connect the new node
    // New node points to the node after the prevNode
    newNode->next = prevNode->next;

    // newNode previous must be pointed to the prevNode
    newNode->prev = prevNode;

    //Check if there is a node after prevNode
    // make it point back to the newNode
    if (prevNode->next != nullptr) {
        prevNode->next->prev = newNode;
    }

    // prevNode will be pointed to the newNode
    prevNode->next = newNode;
}

// General Insertion
std::cout << "\nGeneral Insertion (Insert 6 after P):" << std::endl;
dllGeneralInsertion(second, '6');
dllTraverse(head);
```

A new node containing 6 is inserted after the node with data P. Both the next and prev pointers of the surrounding nodes are updated to keep the list properly connected in both directions.

5. Delete the node

```
template <typename T>
void dllDelete(T findData, doubleList<T>** head) {

    doubleList<T>* current = *head;

    //Search for the node
    while (current != nullptr && current->data != findData) {
        current = current->next;
    }

    // If value not found
    if (current == nullptr) {
        return;
    }

    // If deleting the head
    if (current == *head) {
        *head = current->next;
    }

    // Update the previous node
    if (current->prev != nullptr) {
        current->prev->next = current->next;
    }

    //Update next node
    if (current->next != nullptr) {
        current->next->prev = current->prev;
    }

    delete current;
}
```

```
// Delete
std::cout << "\nDelete P:" << std::endl;
dllDelete('P', &head);
dllTraverse(head);
```

```
Delete P:
G C 6 E 1
```

The node containing P is removed by reconnecting the neighboring nodes. The next and previous pointers are updated so the deleted nodes are no longer part of the list, then their memory is released.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List

Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple Players take turns one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

SOURCE CODE

```
#include <iostream>
#include <string>

using namespace std;

// Node for each song
struct Song
{
    string title;
    Song* next;
    Song* prev;

    Song(string songTitle)
    {
        title = songTitle;
        next = nullptr;
        prev = nullptr;
    }
}
```

```
};

class Playlist
{
private:
    Song* head;

public:
    Playlist()
    {
        head = nullptr;
    }

    // Add a song to the end of the playlist
    void addSong(string title)
    {
        Song* newSong = new Song(title);

        if (head == nullptr)
        {
            head = newSong;
            newSong->next = newSong;
            newSong->prev = newSong;
            return;
        }

        Song* last = head->prev;

        last->next = newSong;
        newSong->prev = last;
        newSong->next = head;
        head->prev = newSong;
    }

    // Remove a song from the playlist
    void removeSong(string title)
    {
```

```
if (head == nullptr)
{
    cout << "Playlist is empty.\n";
    return;
}

Song* current = head;

do
{
    if (current->title == title)
    {
        if (current->next == current)
        {
            head = nullptr;
        }
        else
        {
            current->prev->next = current->next;
            current->next->prev = current->prev;

            if (current == head)
                head = current->next;
        }

        cout << "\"" << title << "\" was removed.\n";
        delete current;
        return;
    }

    current = current->next;

} while (current != head);

cout << "\"" << title << "\" was not found.\n";
}
```

```
// Display all songs once
void playAll()
{
    if (head == nullptr)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    Song* current = head;

    cout << "\nPlaylist:\n";

    do
    {
        cout << "- " << current->title << endl;
        current = current->next;

    } while (current != head);
}

Song* getHead()
{
    return head;
}

Song* nextSong(Song* current)
{
    return current->next;
}

Song* previousSong(Song* current)
{
    return current->prev;
}
};
```

```
int main()
{
    Playlist playlist;

    // Create playlist
    playlist.addSong("Lifetime - Ben&Ben");
    playlist.addSong("Aura - IV OF SPADES");
    playlist.addSong("Saan? - Maki");
    playlist.addSong("711 - TONEEJAY");

    cout << "Original Playlist";
    playlist.playAll();

    // Add a new song
    cout << "\nAdding \"Seasons - w2e\"...\n";
    playlist.addSong("Seasons - w2e");
    playlist.playAll();

    // Remove a song
    cout << "\nRemoving \"Aura - IV OF SPADES\"...\n";
    playlist.removeSong("Aura - IV OF SPADES");
    playlist.playAll();

    // Demonstrate next and previous
    Song* current = playlist.getHead();

    cout << "\nCurrent Song: " << current->title << endl;

    current = playlist.nextSong(current);
    cout << "Next Song: " << current->title << endl;

    current = playlist.previousSong(current);
    cout << "Previous Song: " << current->title << endl;

    return 0;
}
```

OUTPUT

Original Playlist

Playlist:

- Lifetime - Ben&Ben
- Aura - IV OF SPADES
- Saan? - Maki
- 711 - TONEEJAY

Adding "Seasons - w2e"...

Playlist:

- Lifetime - Ben&Ben
- Aura - IV OF SPADES
- Saan? - Maki
- 711 - TONEEJAY
- Seasons - w2e

Removing "Aura - IV OF SPADES"...

"Aura - IV OF SPADES" was removed.

Playlist:

- Lifetime - Ben&Ben
- Saan? - Maki
- 711 - TONEEJAY
- Seasons - w2e

Current Song: Lifetime - Ben&Ben

Next Song: Saan? - Maki

Previous Song: Lifetime - Ben&Ben

This program demonstrates how a circular doubly linked list can be used to create a simple song playlist. Each song is stored in a node that contains the song title and two pointers: one for the next song and one for the

previous song. The last song is connected back to the first song, making the playlist circular. This allows the playlist to continue looping without reaching the end. The program supports basic playlist operations such as adding a song, removing a song, displaying all songs, and moving to the next or previous song. These operations show how a circular doubly linked list is useful for applications like music players, where users can easily navigate through songs in both directions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used AI to help me how to do the activity, but it is not replaced my critical thinking.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.